ISAM como método de acceso en PostgreSQL 18.6

Informe final: indexación y distribución, bloques internos 0–8

Grupo 9 – Bases de Datos II

Medina Grados, Fabian Esteban
Vargas Llerena, Bryan Aron
Villanueva Valencia, Andy Jeferson
Chavez Quijahuaman, Jorge
Becerra Reymer, Leonardo Sebastian

Proyecto académico – 2026-II

Resumen

Se describe una implementación académica de Indexed Sequential Access Method (ISAM) en C, integrada como método de acceso a índices (*Index Access Method*) de PostgreSQL 18.6. El directorio estático organiza páginas primarias ordenadas y cadenas de desbordamiento; las entradas relacionan claves enteras con identificadores de tupla del heap. Se implementan construcción, igualdad, rango e inserción, y se añaden observabilidad, reorganización, VACUUM y uso del WAL genérico del DBMS. La validación incluye pruebas del núcleo, integración SQL, recuperación ante SIGKILL y una regresión específica de eliminación y VACUUM. La evaluación compara acceso sin índice, B-tree e ISAM con tres tamaños y cinco repeticiones independientes, empleando operaciones fijas, semillas y registros verificables. Se separan estos resultados de los experimentos exploratorios previos. Se incorpora además un escenario de tres nodos PostgreSQL con consultas reales mediante FDW, dos asignaciones, snapshots de catálogos y un modelo explícito de latencias para la Etapa II.

Contenido

Resumen **1**

1 Introducción y motivación **3**

1.1 Alcance y hoja de ruta del equipo 3

2 Fundamentos teóricos y comparación con B-tree **4**

2.1 Costos temporales, espaciales y de E/S 4

3 Diseño físico y layout en C **4**

4 Implementación e integración en PostgreSQL **5**

5 Validación funcional, casos extremos y recuperación **6**

5.1 Regresión de DELETE y VACUUM y su corrección 6

5.2 SIGKILL y WAL genérico 7

6 Metodología y resultados experimentales **7**
6.1 Entorno y diseño controlado 7
6.2 Datasets y cargas 7
6.3 Resultados corregidos y dispersión 8
6.4 Relleno, concentración y REINDEX 11

7 Riesgos técnicos y amenazas a la validez **13**

8 Conclusiones de la Etapa I **13**

9 Diseño distribuido: bloques internos 7 y 8 **14**
9.1 Esquema y generación reproducible 14
9.2 Fragmentación y dos asignaciones 14
9.3 Réplicas reales de catálogos 15
9.4 Papel de B-tree, ISAM y acceso secuencial 15

10 Metodología experimental distribuida **15**
10.1 Latencias de la rúbrica y su simulación 16
10.2 Transferencia y condiciones de ejecución 16

11 Resultados y validación del escenario distribuido **16**
11.1 Interpretación de las mediciones archivadas 16
11.2 Correctitud, réplica e indisponibilidad 19
11.3 Discusión y límites del alcance 19

12 Conclusiones de la entrega final **19**

Contribución propia y recursos utilizados **20**

A Correspondencia con la propuesta inicial, sección 5.5 **20**

B Comandos reproducibles y defensa **21**

Referencias **21**

1 Introducción y motivación

La indexación permite localizar registros sin inspeccionar siempre todas las páginas de una relación. Su conveniencia depende del costo de construcción, selectividad, distribución de accesos, espacio y modificaciones de los datos. El objetivo del proyecto es comprender esas decisiones mediante una estructura propia y su utilización real desde PostgreSQL, conforme a la propuesta inicial del equipo [4] y a las pautas oficiales [5].

Se eligió ISAM porque combina una organización secuencial ordenada con un directorio jerárquico estático y soporta naturalmente igualdad y rango. El diseño permitió observar de forma explícita el compromiso entre un directorio estable y el costo de las cadenas de overflow. B-tree constituye la referencia del DBMS; no se plantea que ISAM lo supere en todos los escenarios.

Los objetivos verificables son: construir y consultar un índice propio; insertar sin perder correspondencia con el heap; ejecutar las operaciones desde el executor mediante callbacks C; reproducir el entorno; y explicar los resultados de una comparación controlada. Se adopta clave `int4` de una columna, con duplicados permitidos. No se pretende ofrecer un componente de producción.

1.1 Alcance y hoja de ruta del equipo

El informe reúne los bloques internos 0–8; la parte inicial conserva la evaluación de indexación y las secciones posteriores documentan la distribución. Las etapas oficiales del curso y los bloques del equipo son organizaciones distintas: el parcial solicita una implementación integrada y una comparación preliminar; los experimentos controlados aquí incluidos adelantan la evaluación del índice.

Bloque	Trabajo	Relación con la entrega
0	Organización del repositorio; separación de núcleo, AM, pruebas y antecedentes	Código actual; `legacy/` solo como referencia
1	Docker, PostgreSQL 18.6 y prototipo de AM	Entorno reconstruible y registro del AM
2	Layout, directorio, construcción y búsqueda en C	Núcleo standalone
3	Inserción y overflow	Pruebas y puntero a la cola
4	Callbacks e integración PostgreSQL	Índices reales, scans y bitmap
5	D1/D2/D3, W1–W4 y automatización	Generadores y cuotas reproducibles
6	Comparación y reorganización	Resultados, relleno y REINDEX
7	Fragmentación, asignación y replicación	Diseño distribuido y contrato de validación
8	Tres nodos y experimentos distribuidos	Implementación, experimentos y evidencia

La propuesta declara fuera del alcance mínimo concurrencia completa, WAL propio, VACUUM especializado y unicidad completa. El uso de mecanismos existentes de PostgreSQL y las pruebas adicionales se presentan como ampliaciones, sin convertir esas pruebas en una garantía general de robustez.

2 Fundamentos teóricos y comparación con B-tree

ISAM utiliza separadores que se determinan al construir el directorio. Las inserciones conservan ese directorio y emplean espacio libre en páginas primarias o páginas de overflow enlazadas. B-tree mantiene una organización dinámica, mediante divisiones y otras operaciones de mantenimiento. Las referencias académicas de la propuesta describen este contraste [1, 2, 6].

Aspecto	ISAM implementado	B-tree de referencia
Directorio	Estático entre reconstrucciones	Dinámico
Inserción	Espacio primario o overflow	Mantenimiento de páginas y divisiones
Búsquedas	Igualdad y rangos por candidatos	Igualdad, rango y capacidades adicionales
Degradación	Cadenas largas bajo concentración	Comportamiento condicionado por ocupación y carga
Reorganización	Reconstrucción total con REINDEX	Mantenimiento dinámico y REINDEX disponible
Capacidades	int4, una columna, sin unicidad ni index-only scan	AM de producción con capacidades más amplias

Tabla 2: Comparación conceptual; no implica superioridad universal de una alternativa.

2.1 Costos temporales, espaciales y de E/S

Sea N el número de entradas, f el fanout y h la altura del directorio. La construcción bottom-up sobre entradas ya ordenadas es $O(N)$. El flujo PostgreSQL recoge y ordena las entradas con `qsort`, por lo que debe incluir el costo del ordenamiento comparativo, del orden de $O(N \log N)$, y memoria $O(N)$. No se debe atribuir complejidad lineal a toda la operación `CREATE INDEX` sobre un heap no ordenado.

El descenso usa búsqueda binaria dentro de cada página INDEX: $O(h \log f)$ comparaciones. Una consulta debe además inspeccionar las páginas y entradas candidatas, incluidas sus cadenas; devolver muchos duplicados o un rango amplio incrementa el costo. El costo de E/S es aproximadamente la altura más las páginas candidatas y de overflow visitadas. Los límites inferior y superior comparten lecturas del directorio si recorren una misma rama.

La inserción en una página primaria puede desplazar entradas, con costo acotado por su capacidad. La cola de overflow evita recorrer toda la cadena para añadir una entrada; la búsqueda de entradas existentes aún puede visitarla completa. REINDEX relee el heap, ordena y reconstruye el índice, con costo global y bloqueo de la operación ordinaria. El espacio crece con N y las páginas de overflow.

3 Diseño físico y layout en C

Las páginas tienen 8 192 bytes (8 KiB). En modo PostgreSQL se reservan 24 bytes para la cabecera del DBMS y 16 bytes para `IsamBlockHeader`. En modo standalone no existe la cabecera PostgreSQL. Las constantes y comprobaciones estáticas se encuentran en `isam_am/core/isam_layout.h`.

- META almacena magic, versión del formato, capacidades, relleno, número de páginas primarias y niveles, raíz y ubicación de los niveles.
- DATA almacena entradas primarias ordenadas y el enlace a su cadena.

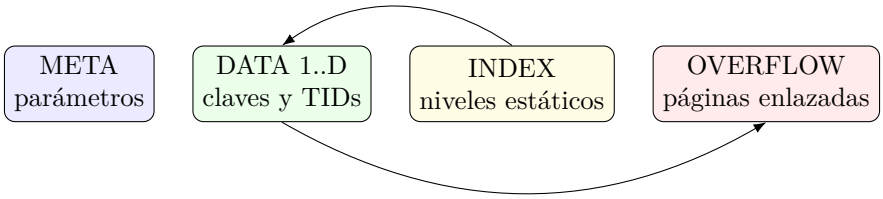


Figura 1: Organización física del índice. Las páginas contienen referencias al heap.

- INDEX contiene pares separador–bloque hijo. Los separadores son estáticos.
- OVERFLOW contiene entradas anexadas y el enlace a la siguiente página; sus entradas no tienen que estar ordenadas.

`IsamEntry` ocupa 12 bytes: clave int32, bloque del heap uint32, offset uint16 y dos bytes reservados. `IsamIndexEntry` ocupa 8 bytes: clave int32 y bloque hijo uint32. El TID identifica la ubicación de la tupla, no su valor completo.

$$C_{\text{DATA,PG}} = \left\lfloor \frac{8\,192 - 24 - 16}{12} \right\rfloor = 679, \qquad f_{\text{INDEX,PG}} = \left\lfloor \frac{8\,192 - 24 - 16}{8} \right\rfloor = 1\,019.$$

En modo standalone las capacidades son 681 y 1 022. El factor de llenado inicial, entre 10 y 100, reserva holgura en DATA; la inserción puede utilizar la capacidad completa antes de crear overflow. La GUC `isam_am.fill_pct` tiene valor predeterminado 80. El formato utiliza el orden de bytes nativo y no se reclama portabilidad arbitraria de archivos físicos entre arquitecturas.

4 Implementación e integración en PostgreSQL

El método de acceso (AM) se registra con `CREATE ACCESS METHOD ... TYPE INDEX` y una clase de operadores para claves `int4`, con las estrategias $<, \leq, =, \geq, >$. La interfaz oficial conecta estas operaciones con `IndexAmRoutine` [3].

Operación	Callback/función	Archivo y responsabilidad
Registro	`isam_handler`	`isam_handler.c`: capacidades y callbacks
Construcción	`isam_ambuild`	`isam_am_build.c`: leer heap, ordenar, construir
Índice inicial	`isam_ambuildempty`	Fork INIT para tablas UNLOGGED
Inserción	`isam_aminsert`	`isam_am_insert.c`: convertir clave/TID y serializar
Abrir/reiniciar scan	`ambeginscan`, `amrescan`	`isam_am_scan.c`: límites y estado diferido
Tuplas/bitmap	`amgettuple`, `amgetbitmap`	Devolver TIDs al executor
Costos	`amcostestimate`	`genericcostestimate`, aproximación simplificada
VACUUM	`ambulkdelete`, `amvacuumcleanup`	Eliminar entradas muertas y actualizar estadísticas
E/S y WAL	`isam_write_block`	`core/isam_io_pg.c`: buffers y GenericXLog

Observabilidad	`isam_index_stats`	Contar entradas y cadenas; `isam_reorg_needed` evalúa umbral

El flujo es SQL → planner → executor → callbacks del AM → núcleo ISAM → buffer manager. Los TIDs devueltos permiten que PostgreSQL consulte el heap y aplique su visibilidad MVCC. La extensión no sustituye el gestor de tuplas del DBMS.

Listing 1: Uso del método de acceso real

```
CREATE EXTENSION IF NOT EXISTS isam_am;
CREATE TEMP TABLE tabla(id int4, payload text);
INSERT INTO tabla
  SELECT id, repeat('p', 40)
  FROM generate_series(1, 2000) AS g(id);
SET isam_am.fill_pct=80;
CREATE INDEX idx ON tabla USING isam_am(id);
EXPLAIN (ANALYZE, BUFFERS)
  SELECT payload FROM tabla WHERE id=680;
SELECT * FROM isam_index_stats('idx');
SELECT isam_reorg_needed('idx',25);
```

La implementación omite NULL en el índice. No soporta unicidad, múltiples columnas, ordenamiento por índice, scans hacia atrás, index-only scan ni construcción paralela. No se reclama soporte de CREATE INDEX CONCURRENTLY. Los insertadores se serializan por el bloqueo de extensión de la relación. El estimador de costos no incorpora una modelación sofisticada de las cadenas.

5 Validación funcional, casos extremos y recuperación

La suite standalone comprueba vacío, igualdad, rangos, límites int32, duplicados, relleno, overflow, persistencia, eliminación y reparación del puntero de cola. Después de añadir la regresión de VACUUM ejecutó **121 452 verificaciones** (3 774 + 117 678), sin fallos.

La integración SQL comprueba diez estados con 21 predicados cada uno: **210 comparaciones** contra seqscan, más aserciones y comprobaciones de planes. Incluye construcción, inserción, UPDATE/DELETE, VACUUM, VACUUM FULL, REINDEX, relleno 100, UNLOGGED, TEMP e índice creado vacío.

5.1 Regresión de DELETE y VACUUM y su corrección

El caso `tests/sql/caso-vacuum.sql` construye dos páginas primarias llenas, inserta otra fila de clave 680 en overflow, borra solamente la original y ejecuta VACUUM. La versión anterior devolvía una fila por seqscan y cero por ISAM: el escaneo usaba la primera clave primaria superviviente como si fuera el separador estático. Al compactar la página, ese mínimo aumentaba, mientras que overflow conservaba una clave menor.

La corrección conserva VACUUM y modifica el lector. `isam_index_range_blocks` calcula las páginas candidatas a partir de separadores estáticos. El scan usa el último candidato como condición de finalización, examina el overflow de cada página visitada y nunca descarta una cadena basándose solo en el mínimo primario mutable. Los caminos del directorio se comparten cuando es posible.

La regresión exige la fila superviviente antes de REINDEX por Index Scan y Bitmap Scan, comprueba rangos inclusivos/exclusivos y ausencia, y verifica que REINDEX mantenga el resultado.

El núcleo incorpora el caso equivalente y persistencia. La igualdad habitual en la prueba de un millón de claves conserva tres lecturas de bloque; un rango de 10 000 claves conserva las 18 lecturas observadas en esa prueba, sin atribuir esas lecturas standalone a E/S física del servidor.

5.2 SIGKILL y WAL genérico

Las páginas se escriben mediante GenericXLog: las nuevas utilizan imagen completa y las existentes deltas. No se implementa un formato de WAL independiente. `make crash` mata el contenedor con SIGKILL, reinicia y comprueba el índice contra el heap, además de la pérdida esperada de datos UNLOGGED. `make crash-load` inyecta la caída con seis clientes insertando y verifica conteo y suma tras recuperación. Ambas pruebas pasaron en el entorno Docker 18.6 corregido.

Estas pruebas no demuestran corrección para todos los entrelazamientos de operaciones concurrentes ni preservación de transacciones no confirmadas. Las mediciones históricas de volumen WAL de 160 y 57 MB no se incorporan como resultados nuevos porque los CSV aportados no incluyen sus logs. Deltas de GenericXLog no equivalen a medir `wal_compression`.

6 Metodología y resultados experimentales

6.1 Entorno y diseño controlado

El entorno de pruebas utilizó una compilación controlada de PostgreSQL 18.6. La imagen base y las fuentes se identificaron mediante sumas de comprobación SHA-256; las descargas conservaron TLS y los paquetes del sistema su verificación de firmas. Una suma SHA-256 permite verificar contenido, pero no es una firma digital. Los metadatos guardan la versión y los parámetros de cada corrida.

La ejecución registrada tenía una cuota efectiva de cuatro CPU equivalentes y un límite de memoria cgroup de 32 GiB (34 359 738 368 bytes), según `docs/results/resource-limits.json`. Estos límites restringen recursos; no garantizan aislamiento frente a otras actividades del sistema anfitrión.

Se desactivaron JIT y el paralelismo mediante los parámetros siguientes:

- Consultas: `max_parallel_workers_per_gather=0`.
- Construcción: `max_parallel_maintenance_workers=0`.

Se mantuvieron habilitados `fsync`, `synchronous_commit` y `full_page_writes`. El benchmark principal utilizó cuatro clientes y cuatro hilos de `pgbench`; el experimento adicional de overflow utilizó un cliente. Las tablas presentan la mediana de las latencias medias por réplica, no percentiles de consultas individuales.

Antes de cada alternativa y repetición se recargó la tabla, se validaron cantidad, unicidad y dominio, y se ejecutó `VACUUM ANALYZE`. El procedimiento utiliza datos previamente leídos y no constituye una prueba de caché fría. Se mantuvo el mismo protocolo y se bloquearon demostraciones y pruebas de caída concurrentes sobre el mismo contenedor. No se afirma que el estado de caché fuera idéntico ni que se eliminaran todas las interferencias externas.

6.2 Datasets y cargas

La unicidad D2 se obtiene por construcción, sin muestreo con reemplazo. D3 incluye `d2.sql`; el sesgo se aplica al elegir claves de consulta, no al duplicar filas. D4/D5 y búsquedas por prefijo

ID	Definición	Tamaños
D1	Claves únicas $1, \ldots, N$, carga secuencial	100 000; 500 000; 1 000 000
D2	Permutación aleatoria única de $1, \ldots, N$, semilla SQL 0.42	100 000; 500 000; 1 000 000
D3	Exactamente los datos D2; consultas Zipf con parámetro 1.1	100 000; 500 000; 1 000 000

Tabla 4: Definiciones alineadas con la rúbrica; distribución de accesos y de valores se distinguen.

quedan fuera del alcance int4 de la propuesta.

Carga	Operación	Cuota por réplica
W1	Igualdad sobre claves existentes	10 000 consultas
W2	Igualdad sobre claves ausentes en $(N, 2N]$	10 000 consultas
W3	Inserción uniforme de claves del dominio, duplicados permitidos	$N/10$ filas
W4	Rango inclusivo de amplitud $\lfloor N/1\,000 \rfloor$	1 000 consultas
Posteriores	Repetir W1 y W4 tras W3	10 000 y 1 000 consultas

Tabla 5: Operaciones fijas; W3 aumenta la cantidad de filas exactamente un 10%.

Cada medición tiene cinco repeticiones independientes. En cada réplica se vuelve a medir `CREATE INDEX`; el tiempo no se copia de una construcción anterior. La semilla de `pgbench` es $42\,000 + r$, donde r es el número de réplica, idéntica entre alternativas. Cada cliente ejecuta una cuota fija mediante `-t`; no se usan duraciones fijas que produzcan cantidades distintas de inserciones. Se verifica el conteo final y la huella del multiconjunto insertado. La independencia es del estado de cada corrida; no presupone independencia estadística absoluta respecto de cachés o variaciones del host.

En el barrido principal W3 utiliza una transacción por fila insertada. Los tamaños físicos se convierten a MiB, donde $1\,\text{MiB} = 2^{20}$ bytes.

Las consultas agregan `payload` para requerir acceso al heap. Esta decisión controla una comparación específica; excluye deliberadamente la ventaja de index-only scan disponible en ciertos usos de B-tree y no representa todos sus casos de aplicación. El planner puede elegir su plan; los benchmarks no fuerzan ISAM. Los buffers registrados corresponden a predicados de muestra, no al promedio de todas las consultas ni directamente a lecturas físicas.

6.3 Resultados corregidos y dispersión

Los resultados se importan del CSV corregido y de sus metadatos con estado `completed`. El generador de tablas rechaza una cuadrícula incompleta, transacciones fallidas o cuotas incorrectas. Se reporta mediana y rango intercuartílico (IQR) entre las cinco réplicas. El IQR es una medida descriptiva, no un intervalo de confianza ni una prueba de equivalencia estadística.

La corrida verificada contiene 810 observaciones, 135 cargas independientes y 90 construcciones de índices. El IQR se indica entre corchetes. Todos los conteos y huellas entre alternativas fueron verificados.

Versión registrada: PostgreSQL 18.6 on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit.

En W1, el cociente de medianas TPS ISAM/B-tree se encuentra entre 0.976 y 1.362. Es una descripción de esta corrida y de este protocolo, sin prueba de equivalencia estadística. Las tablas de construcción distinguen cinco reconstrucciones efectivas; los tiempos repetidos en filas de otros

Tabla 6: Construcción (ms): mediana [IQR], y tamaño inicial (MiB).

Datos	N	B-tree ms	ISAM ms	B-tree MiB	ISAM MiB
D1	100 000	24.000 [3.511]	11.588 [1.244]	2.16	1.46
D1	500 000	91.447 [10.039]	47.667 [2.815]	10.73	7.21
D1	1 000 000	172.407 [5.425]	94.946 [2.221]	21.45	14.42
D2	100 000	30.099 [1.952]	22.221 [0.363]	2.16	1.46
D2	500 000	168.088 [29.991]	114.604 [6.054]	10.73	7.21
D2	1 000 000	411.475 [34.180]	235.824 [8.886]	21.45	14.42
D3	100 000	29.738 [1.553]	23.789 [1.163]	2.16	1.46
D3	500 000	169.060 [23.696]	114.744 [1.551]	10.73	7.21
D3	1 000 000	439.200 [149.195]	243.502 [13.298]	21.45	14.42

Tabla 7: Igualdad exitosa W1: TPS y latencia media por réplica (ms), resumidos con mediana [IQR].

Datos	N	Sin TPS	B-tree TPS	ISAM TPS	Sin ms	B-tree ms	ISAM ms
D1	100 000	1 294 [32]	44 475 [3 807]	60 563 [14 290]	3.090 [0.078]	0.090 [0.007]	0.066 [0.018]
D1	500 000	231 [6]	46 510 [9 759]	47 825 [1 752]	17.294 [0.447]	0.086 [0.017]	0.084 [0.004]
D1	1 000 000	113 [0]	40 737 [6 193]	43 941 [3 986]	35.261 [0.146]	0.098 [0.014]	0.091 [0.008]
D2	100 000	1 293 [53]	46 838 [4 047]	50 871 [3 313]	3.093 [0.128]	0.085 [0.007]	0.079 [0.005]
D2	500 000	238 [7]	47 625 [6 192]	51 102 [14 941]	16.802 [0.475]	0.084 [0.011]	0.078 [0.026]
D2	1 000 000	112 [1]	44 296 [8 910]	46 604 [6 655]	35.661 [0.461]	0.090 [0.016]	0.086 [0.011]
D3	100 000	1 309 [56]	48 374 [12 018]	47 202 [6 748]	3.056 [0.126]	0.083 [0.020]	0.085 [0.011]
D3	500 000	234 [10]	40 685 [11 400]	48 216 [4 895]	17.124 [0.793]	0.098 [0.022]	0.083 [0.009]
D3	1 000 000	113 [5]	40 830 [7 067]	48 916 [6 131]	35.302 [1.502]	0.098 [0.014]	0.082 [0.011]

Tabla 8: Igualdad ausente W2: TPS y latencia media por réplica (ms), resumidos con mediana [IQR].

Datos	N	Sin TPS	B-tree TPS	ISAM TPS	Sin ms	B-tree ms	ISAM ms
D1	100 000	1 297 [30]	57 594 [7 668]	52 896 [15 054]	3.085 [0.072]	0.069 [0.010]	0.076 [0.019]
D1	500 000	235 [8]	60 508 [11 393]	47 604 [563]	17.057 [0.638]	0.066 [0.015]	0.084 [0.001]
D1	1 000 000	115 [3]	50 198 [14 630]	48 665 [3 857]	34.675 [0.946]	0.080 [0.019]	0.082 [0.006]
D2	100 000	1 282 [49]	58 058 [11 251]	54 680 [7 145]	3.120 [0.117]	0.069 [0.013]	0.073 [0.010]
D2	500 000	235 [3]	51 808 [9 671]	46 983 [4 646]	16.986 [0.212]	0.077 [0.014]	0.085 [0.008]
D2	1 000 000	113 [6]	46 799 [4 304]	49 234 [6 397]	35.276 [1.935]	0.085 [0.008]	0.081 [0.011]
D3	100 000	1 293 [45]	47 893 [9 921]	54 561 [3 910]	3.094 [0.105]	0.084 [0.015]	0.073 [0.005]
D3	500 000	229 [8]	59 343 [2 678]	56 849 [5 780]	17.492 [0.604]	0.067 [0.003]	0.070 [0.008]
D3	1 000 000	111 [1]	49 127 [10 285]	48 417 [4 287]	36.064 [0.200]	0.081 [0.019]	0.083 [0.008]

Tabla 9: Rango W4: TPS y latencia media por réplica (ms), resumidos con mediana [IQR].

Datos	N	Sin TPS	B-tree TPS	ISAM TPS	Sin ms	B-tree ms	ISAM ms
D1	100 000	1 140 [54]	33 228 [3 316]	30 983 [3 141]	3.510 [0.163]	0.120 [0.013]	0.129 [0.012]
D1	500 000	217 [4]	20 868 [1 110]	22 241 [580]	18.433 [0.375]	0.192 [0.010]	0.180 [0.005]
D1	1 000 000	111 [4]	16 900 [1 450]	14 930 [2 923]	36.171 [1.346]	0.237 [0.021]	0.268 [0.053]
D2	100 000	1 076 [66]	20 245 [4 650]	22 681 [2 123]	3.718 [0.223]	0.198 [0.038]	0.176 [0.018]
D2	500 000	204 [5]	8 286 [837]	7 926 [575]	19.641 [0.512]	0.483 [0.049]	0.505 [0.037]
D2	1 000 000	98 [6]	4 250 [471]	4 294 [327]	40.861 [2.541]	0.941 [0.098]	0.931 [0.075]
D3	100 000	1 112 [67]	21 568 [1 977]	22 506 [5 215]	3.598 [0.213]	0.185 [0.018]	0.178 [0.040]
D3	500 000	202 [2]	9 163 [1 126]	10 037 [1 288]	19.817 [0.208]	0.437 [0.059]	0.399 [0.055]
D3	1 000 000	97 [5]	4 958 [900]	5 064 [107]	41.383 [2.049]	0.807 [0.140]	0.790 [0.017]

Tabla 10: Inserción W3: TPS y latencia media por réplica (ms), resumidos con mediana [IQR].

Datos	N	Sin TPS	B-tree TPS	ISAM TPS	Sin ms	B-tree ms	ISAM ms
D1	100 000	9 792 [699]	9 750 [259]	9 159 [1 218]	0.408 [0.030]	0.410 [0.010]	0.437 [0.055]
D1	500 000	10 087 [222]	9 681 [201]	9 636 [96]	0.397 [0.009]	0.413 [0.008]	0.415 [0.004]
D1	1 000 000	10 054 [659]	9 700 [680]	9 206 [706]	0.398 [0.026]	0.412 [0.030]	0.435 [0.032]
D2	100 000	10 097 [776]	9 980 [444]	8 838 [578]	0.396 [0.029]	0.401 [0.017]	0.453 [0.029]
D2	500 000	9 803 [619]	10 239 [282]	10 098 [348]	0.408 [0.025]	0.391 [0.011]	0.396 [0.014]
D2	1 000 000	9 957 [419]	9 919 [688]	9 451 [347]	0.402 [0.017]	0.403 [0.029]	0.423 [0.015]
D3	100 000	10 664 [572]	10 539 [3 165]	9 426 [500]	0.375 [0.021]	0.380 [0.161]	0.424 [0.021]
D3	500 000	10 404 [366]	9 921 [596]	9 803 [698]	0.384 [0.014]	0.403 [0.025]	0.408 [0.030]
D3	1 000 000	9 830 [438]	10 103 [420]	9 636 [89]	0.407 [0.018]	0.396 [0.017]	0.415 [0.004]

Tabla 11: Estado después de insertar exactamente un 10% de filas; tamaños en MiB.

Datos	N inicial	Filas finales	B-tree MiB	ISAM MiB	Overflow ISAM	Cadena ISAM
D1	100 000	110 000	2.17	1.46	0 [0]	0 [0]
D1	500 000	550 000	10.80	7.21	0 [0]	0 [0]
D1	1 000 000	1 100 000	21.60	14.42	0 [0]	0 [0]
D2	100 000	110 000	2.17	1.46	0 [0]	0 [0]
D2	500 000	550 000	10.80	7.21	0 [0]	0 [0]
D2	1 000 000	1 100 000	21.59	14.42	0 [0]	0 [0]
D3	100 000	110 000	2.17	1.46	0 [0]	0 [0]
D3	500 000	550 000	10.80	7.21	0 [0]	0 [0]
D3	1 000 000	1 100 000	21.59	14.42	0 [0]	0 [0]

workloads de la misma réplica no se cuentan como reconstrucciones adicionales.

La mediana de construcción de ISAM fue menor que la de B-tree en las nueve combinaciones de datos y tamaño. El cociente de tiempos ISAM/B-tree varió entre 0.483 y 0.800; no se infiere la misma ventaja para otros entornos o construcción paralela.

El ahorro inicial de tamaño de ISAM frente a B-tree estuvo entre 32.25 y 32.82%. Este resultado es compatible con entradas compactas de 12 bytes y ausencia de un identificador de elemento (*item identifier*) por entrada en el layout ISAM. Se refiere a claves iniciales únicas y a los rellenos configurados; no demuestra la misma relación con duplicados arbitrarios.

En rangos el cociente de TPS ISAM/B-tree varió entre 0.883 y 1.120; en inserciones por fila, entre 0.886 y 0.995. La organización física del heap, el plan elegido y el costo de recuperar payload influyen junto con el índice. No se atribuye toda diferencia a una única causa sin instrumentación adicional.

Después de la inserción uniforme del 10%, el máximo de páginas de overflow observado en el barrido principal fue 0. La holgura inicial del 20% puede absorber esta carga; para exponer el caso desfavorable se necesita el experimento concentrado siguiente. Una carga sin cadenas largas no demuestra inmunidad a overflow.

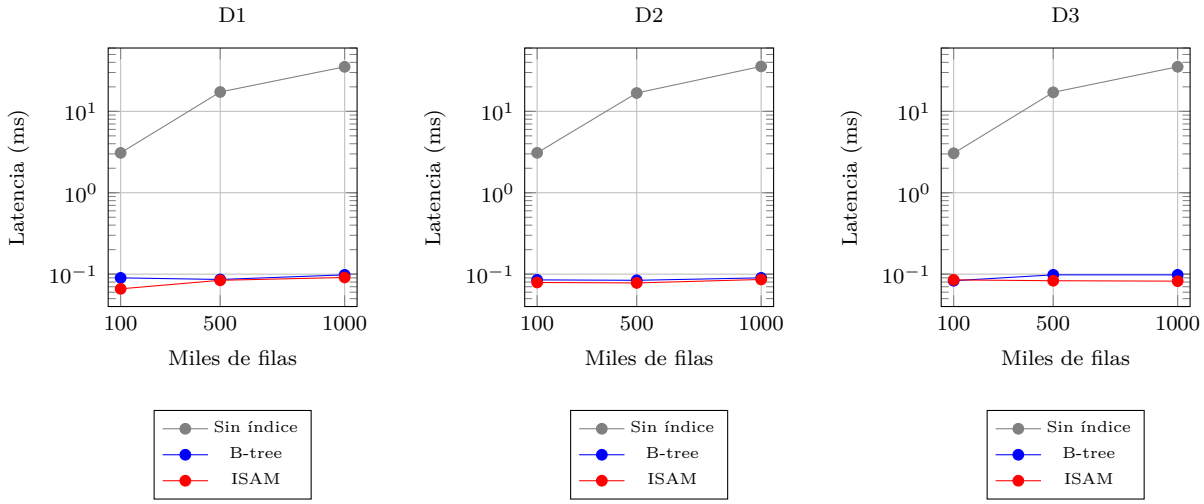


Figura 2: Igualdad exitosa: mediana de la latencia media por réplica y escalabilidad observada.

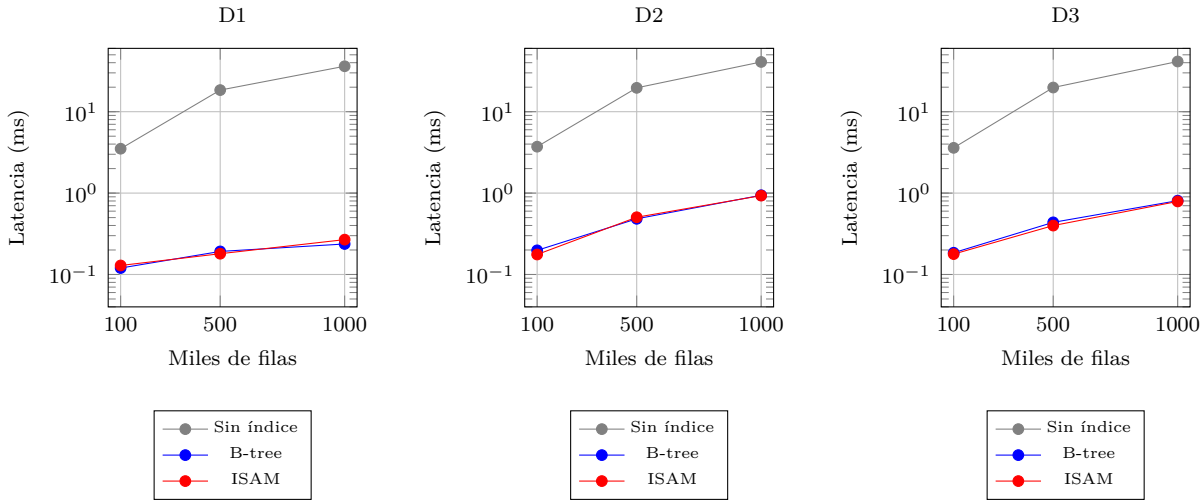


Figura 3: Búsqueda por rango: mediana de la latencia media por réplica y escalabilidad observada.

6.4 Relleno, concentración y REINDEX

El experimento adicional carga D1 y concentra lotes de inserciones en una ventana de 100 claves. Compara B-tree, ISAM con rellenos 60/80/100 y una política ISAM80 con umbral de 25%. Cada trayectoria se recarga y repite cinco veces. Las lecturas calientes usan cuotas y semillas fijas. REINDEX vuelve a fijar el relleno de la configuración para no cambiarlo silenciosamente entre sesiones. La política se ejecuta desde el runner; su costo y pausas son parte del compromiso de mantenimiento.

Este experimento adicional inserta cada lote mediante un INSERT SELECT en una transacción. Sus tiempos de inserción por lote no son directamente comparables con el TPS por transacción de W3; ambos protocolos se identifican por separado.

El experimento adicional comprende $N = 100\,000$ filas iniciales, cinco trayectorias independientes por configuración, seis lotes de 20 000 filas y 1 000 consultas calientes por punto: 195 observaciones en total.

Los costos de la política se suman por réplica antes de calcular la mediana. Una latencia final cercana a B-tree no significa regresar a la latencia inicial: las inserciones aumentan la cantidad de resultados. La eliminación de overflow tampoco implica siempre menor tamaño físico.

Tabla 12: Lectura caliente inicial, final y después de REINDEX: mediana [IQR].

Configuración	Inicial ms	Final ms	Cadena final	Tras REINDEX ms	REINDEX ms
btree	0.054 [0.006]	0.531 [0.010]	0 [0]	0.504 [0.063]	63.779 [1.664]
isam60	0.053 [0.001]	0.757 [0.034]	110 [1]	0.521 [0.099]	49.909 [3.797]
isam80	0.057 [0.006]	0.980 [0.003]	177 [0]	0.547 [0.016]	48.335 [1.321]
isam100	0.052 [0.010]	0.951 [0.023]	177 [0]	0.533 [0.036]	44.606 [0.418]
isam80t25	0.049 [0.003]	0.551 [0.024]	0 [0]	–	–

Tabla 13: Costo acumulado por trayectoria (medianas, ms) antes del REINDEX final de comparación.

Configuración	Inserción	Política REINDEX	Suma de costos	MiB finales
btree	178.466	0.000	178.466	2.95
isam60	657.916	0.000	657.916	3.32
isam80	654.008	0.000	654.008	2.84
isam100	661.393	0.000	661.393	2.55
isam80t25	859.609	182.187	1 044.330	3.19

ISAM80 pasó de una mediana de 0.057 ms a 0.980 ms, con cadena final mediana de 177 páginas. Tras REINDEX registró 0.547 ms: cambio de 44.18% respecto del estado degradado. La mejora combina eliminación de cadenas y reorganización; la lectura de más filas y páginas del heap persiste.

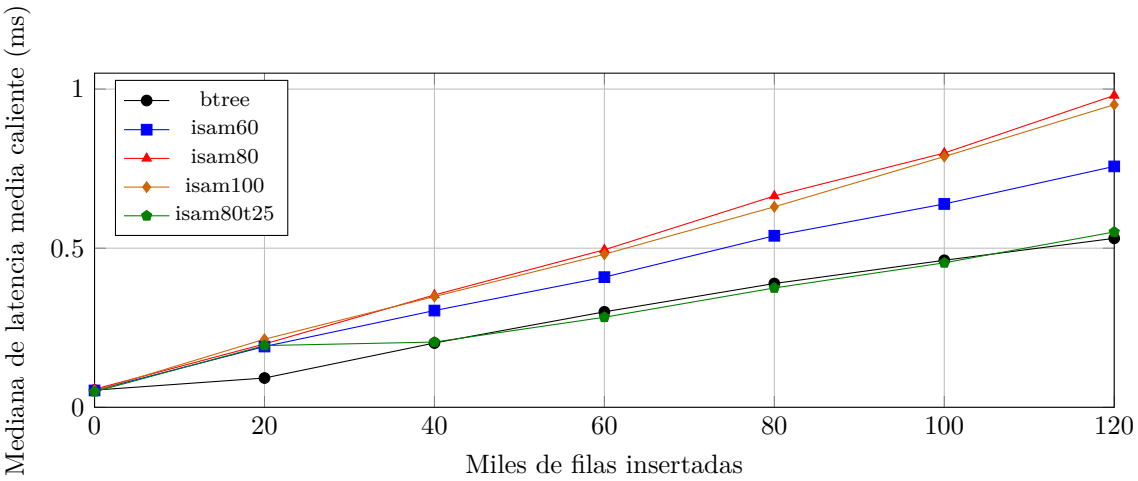


Figura 4: Latencia caliente por lote. Los puntos de la política de umbral incluyen el estado posterior a reorganizar.

No se mezclan aquí los CSV exploratorios anteriores con los corregidos. Esos CSV tenían D2 con duplicados, D3 con otra distribución y tres repeticiones. La comparación de espacio bajo duplicados sigue siendo una pregunta útil, pero debe presentarse como escenario adicional identificado, no como D2 oficial.

7 Riesgos técnicos y amenazas a la validez

Riesgo de la propuesta	Impacto	Mitigación aplicada o límite
API interna PostgreSQL	Incompatibilidad y errores de callbacks	Versión 18.6 exacta, PGXS del mismo commit, integración y planes
Concentración de overflow	Latencia y crecimiento de cadenas	Métricas SQL, relleno y REINDEX; evaluar también su costo
Desarrollo de todas las capas	Complejidad y regresiones	Núcleo separado, pruebas C/SQL y regresión específica de VACUUM
Comprensión desigual	Dificultad en defensa individual	Guía, lectura cruzada y ensayo de todas las capas
Estado mutable entre réplicas	Comparación injusta	Recarga y construcción independiente; cuotas y huellas
Host compartido y cachés	Variabilidad de tiempos	Protocolo explícito, metadatos, mediana e IQR; no afirmar universalidad

El uso de un tipo int4 y payload de 40 bytes limita la generalización a otras claves y tamaños de fila. Los rangos pueden devolver cardinalidades diferentes tras insertar; interpretar el costo del resultado y del heap junto al costo del índice. La muestra de cinco réplicas satisface el mínimo solicitado, pero no demuestra equivalencia universal. El host, contenedores y paquetes pueden variar entre máquinas; la versión fuente y parámetros son verificables, sin prometer reproducibilidad binaria de todos los paquetes del sistema.

Las alternativas se ejecutan en un orden fijo por réplica. El orden, la actividad del host y la duración corta de algunas mediciones indexadas pueden introducir variabilidad; se informa dispersión y no se presentan diferencias pequeñas como significativas sin análisis estadístico adicional.

La reorganización es total y bloqueante en el flujo probado. No hay búsqueda index-only, unicidad ni concurrencia completa. El puntero a la cola evita recorrer la cadena al insertar, pero no reutiliza automáticamente huecos intermedios de overflow liberados por VACUUM. Las pruebas ante SIGKILL son evidencia de los casos ejecutados, no una certificación de almacenamiento de producción.

8 Conclusiones de la Etapa I

La estructura implementada se utiliza desde PostgreSQL mediante un AM propio, con construcción, igualdad, rango e inserción en C. La separación de layout, directorio, escaneo y adaptador del DBMS permite comprobar el mismo núcleo en standalone y en relaciones reales. La regresión de VACUUM mostró que un mínimo primario mutable no sustituye un separador estático; su corrección preserva la fila superviviente y se incorpora a las pruebas de la entrega.

La evaluación corregida distingue valores aleatorios únicos de accesos sesgados y mantiene cantidades de operaciones iguales. Las conclusiones cuantitativas deben basarse en las tablas generadas y su dispersión, sin extrapolar cifras de una máquina a todo contexto. El directorio estático y el overflow explican los compromisos de ISAM: puede resultar apropiado para datos estables, mientras que la concentración de inserciones requiere considerar crecimiento de cadenas, latencia y costo de reconstrucción.

9 Diseño distribuido: bloques internos 7 y 8

El escenario aplica las secciones 6.3 y 6.3.1 de la rúbrica. Se utilizan tres servidores PostgreSQL 18.6 reales, A, B y C, en contenedores Docker Linux con volúmenes independientes. El diseño está especificado en `docs/bloque-7.md`; su implementación y protocolo están en `docs/bloque-8.md`. La extensión `postgres_fdw` ejecuta consultas reales entre esos servidores [7].

9.1 Esquema y generación reproducible

El esquema representa una tienda. `customers` contiene 3 000 clientes, `products` contiene 1 000 productos, `orders` contiene un millón de pedidos y `order_items` contiene un detalle por pedido. Las cuatro relaciones están vinculadas por claves foráneas. Cada nodo almacena copias completas de clientes y productos. Un pedido y su detalle pertenecen al mismo propietario, por lo que sus restricciones se comprueban localmente.

Para el identificador i, el cliente es $1 + ((i-1) \bmod 3000)$ y su región es $\lfloor (customer_id - 1)/1000 \rfloor$. Cantidad, precio histórico e importe se derivan de expresiones enteras reproducibles. Los generadores SQL se encuentran en `distributed/db.py`. El precio guardado en el detalle corresponde a la factura; refrescar el catálogo no modifica importes históricos.

Relación	Vínculo	Ubicación
Clientes	PK id; región 0/1/2	Réplica completa en A/B/C
Productos	PK id	Fuente A; snapshots en B/C
Pedidos	PK id; FK cliente	Fragmento según estrategia
Detalles	PK/FK pedido; FK producto	Mismo propietario del pedido

Tabla 15: Cuatro relaciones vinculadas. Las cantidades lógicas no suman las copias replicadas.

9.2 Fragmentación y dos asignaciones

La estrategia `regional` asigna A=región 0, B=región 1 y C=región 2. La estrategia `hash` asigna según:

$$h(i) = \big((1103515245\, i + 12345) \bmod 2147483647\big) \bmod 3.$$

Esta mezcla modular determinista evita correlacionar directamente los pedidos de un cliente con un único residuo. No se utiliza con fines criptográficos. Los cálculos SQL usan enteros de 64 bits para el producto intermedio. Los predicados son completos y disjuntos; sus CHECK locales, las PK y la validación global de identificadores permiten comprobar la reconstrucción. Los detalles heredan el propietario del pedido.

La asignación regional favorece solicitudes procedentes de la región del cliente. La asignación hash distribuye sus pedidos entre nodos y requiere consultar varios fragmentos cuando solo se conoce el cliente. Se comparan las mismas operaciones lógicas; el encaminamiento cambia según la asignación.

Cada origen conserva tablas locales en `dist` y tablas extranjeras en `remote_a`, `remote_b` y `remote_c` para los demás nodos. Las vistas `dist.all_orders` y `dist.all_order_items` reconstruyen las relaciones con UNION ALL. El cliente dirige escrituras a un único propietario. No se implementan confirmación atómica global ni un snapshot MVCC común entre servidores. Los ensayos ejecutan operaciones secuenciales y comparan cada resultado con una referencia que conoce las escrituras previas.

9.3 Réplicas reales de catálogos

Los clientes permanecen inmutables durante la carga. Los productos se copian desde A a B/C mediante snapshots reales y refresco explícito. El origen lee una versión consistente en REPEATABLE READ; cada réplica publica el snapshot y su número de versión en una transacción local. Las versiones identifican snapshots publicados; A puede tener modificaciones todavía no publicadas.

Se demuestra el retraso: una actualización confirmada en A conserva el precio anterior en B/C hasta ejecutar `refresh_products`. Después se comprueban precios, versiones y huellas iguales. No se trata de replicación lógica nativa, streaming ni consenso. El mecanismo no propaga eliminaciones de productos, no permite escritores múltiples y no realiza promoción automática. Los pedidos no tienen réplicas: su fragmento queda inaccesible si su propietario se detiene. Las copias de catálogos sí permiten lecturas locales en nodos sanos.

9.4 Papel de B-tree, ISAM y acceso secuencial

Todas las alternativas conservan PK B-tree para unicidad, FK, rangos por id y actualizaciones dirigidas. La comparación `none/btree/isam` afecta únicamente al índice secundario `orders.customer_id`; `none` significa ausencia de ese secundario. ISAM no se utiliza para imponer unicidad. Cada nodo construye índices sobre su propio fragmento. Sus TIDs identifican tuplas del heap local; no se copian TIDs ni páginas de índice entre nodos. FDW transmite consultas y valores, y el executor del servidor propietario utiliza sus propios callbacks e índices.

Las lecturas por cliente agregan `count(amount_cents)` y `sum(status)`, por lo que deben recuperar columnas del heap. El índice secundario puede ayudar a filtrar esas lecturas. Los planes normales quedan guardados; se añade un plan forzado como diagnóstico separado para comprobar capacidades del AM. La carga medida deja elegir al planner. Se convierten los parámetros a int4 porque la clase de operadores académica no implementa comparaciones cruzadas int4/int2. Los costos de FDW y del modelo de latencia pueden dominar diferencias locales entre índices; no se atribuye cada diferencia exclusivamente a la estructura.

10 Metodología experimental distribuida

El experimento principal combina dos asignaciones, tres índices secundarios, dos perfiles y cinco repeticiones: 60 ensayos independientes, cada uno con un millón de pedidos y un millón de detalles. La recarga y la reconstrucción se repiten para cada condición. Una huella verifica que los datos iniciales son iguales entre alternativas.

Cada ensayo ejecuta exactamente 100 operaciones: 50 lecturas por cliente, 20 lecturas por rango de identificadores y 30 actualizaciones del estado de pedidos. Las actualizaciones se confirman individualmente. La generación usa semilla $42000 + r$ y conserva una huella de la secuencia. El perfil equilibrado elige origen y región con pesos 1:1:1; el perfil concentrado usa 1:1:8 hacia C. Se registra la proporción realizada; una muestra finita puede diferir del 80%. El cliente es secuencial, por lo que no se infiere saturación concurrente.

Un control adicional repite cinco veces cada asignación con ISAM y perfil equilibrado, sin espera artificial: diez ensayos. Las pruebas de aceptación oficiales repiten cinco veces cada asignación, también con un millón de pedidos. Los diagnósticos pequeños se etiquetan como `smoke` y no se incorporan a las tablas oficiales.

10.1 Latencias de la rúbrica y su simulación

Se interpretan A–B=20 ms, A–C=40 ms y B–C=25 ms como un costo de ida y vuelta por destino remoto utilizado. El modo `model` suma esos costos y espera en el cliente antes de ejecutar el SQL real. Por ejemplo, consultar B y C desde A añade 60 ms. El modo `base` añade cero espera y conserva el mismo SQL. La matriz es simétrica; una operación local recibe cero costo modelado.

Se guardan separadamente la espera solicitada, el tiempo real de ejecución y recuperación del resultado, y el tiempo completo que incluye la espera. La red Docker no se modifica. Esta simulación no representa todas las rondas de FDW, congestión, ancho de banda, pérdida de paquetes ni retrasos por cada fila. El costo modelado no constituye una medición física de los enlaces.

10.2 Transferencia y condiciones de ejecución

Se leen los contadores RX/TX de eth0 de cada contenedor antes y después de la carga, excluyendo recarga, planes de diagnóstico y validación. Incluyen tráfico cliente, FDW y protocolo. Sumar RX y TX puede contar ambos extremos de un enlace; no equivale a bytes útiles únicos. El tamaño JSON del resultado se registra por separado como métrica lógica.

Se registran versión, configuración, identificadores de imagen Docker, cuotas CPU/memoria, semillas y hashes de fuentes y artefactos. Se mantienen fsync, confirmación síncrona y WAL de páginas completas. Se desactivan JIT y paralelismo para el protocolo documentado. Las validaciones leen previamente los datos; no se afirma caché fría ni estados idénticos. El orden de alternativas es fijo, el host puede tener otras actividades y algunas medidas SQL son cortas. Cinco repeticiones y el IQR describen dispersión sin demostrar significancia.

11 Resultados y validación del escenario distribuido

Se validaron 70 ensayos independientes con N=1 000 000 pedidos y cinco repeticiones por condición. Cada ensayo contiene 100 operaciones (70 lecturas y 30 escrituras).

Se reporta mediana [IQR] entre réplicas. SQL real mide el tiempo observado de ejecutar y recuperar el resultado; total incluye la espera de latencia simulada. RX/TX incluyen tráfico de cliente y FDW, y no deben sumarse como bytes útiles únicos.

Las 10 validaciones oficiales de réplica e indisponibilidad terminaron correctamente. A/B conservaron lecturas y una escritura local; el total global y la escritura hacia C fallaron con C detenido. Después de reiniciar C se verificaron los datos y la única escritura confirmada.

11.1 Interpretación de las mediciones archivadas

En el perfil equilibrado con el modelo de latencias, ISAM registró medianas de latencia media total de 12.506 ms con asignación regional y 47.461 ms con hash. El componente SQL observado fue de 1.362 y 2.648 ms, respectivamente. Los valores proceden de cinco réplicas por condición. La diferencia entre total y SQL incluye la espera añadida por el modelo; no constituye una medición física de los enlaces.

Tabla 16: Resultados distribuidos: mediana [IQR] de la latencia media por ensayo; transferencia mediana por ensayo.

Modo	Estrategia	Índice secundario	Perfil	Total ms	SQL real ms	Modelo ms	RX MiB	TX MiB
base	hash	isam	balanced	1.583 [0.130]	1.582 [0.130]	0.000 [0.000]	0.211	0.183
base	regional	isam	balanced	1.016 [0.274]	1.014 [0.273]	0.000 [0.000]	0.076	0.060
model	hash	btree	balanced	47.559 [0.464]	2.778 [0.185]	44.700 [0.500]	0.235	0.202
model	hash	btree	hot_c	52.541 [1.051]	2.582 [0.118]	49.900 [1.050]	0.235	0.202
model	hash	isam	balanced	47.461 [0.673]	2.648 [0.088]	44.700 [0.500]	0.235	0.202
model	hash	isam	hot_c	52.760 [0.435]	2.718 [0.126]	49.900 [1.050]	0.235	0.201
model	hash	none	balanced	65.491 [0.248]	20.459 [0.856]	44.700 [0.500]	0.235	0.202
model	hash	none	hot_c	71.925 [1.386]	21.496 [0.313]	49.900 [1.050]	0.235	0.202
model	regional	btree	balanced	12.406 [0.145]	1.280 [0.098]	11.200 [0.250]	0.084	0.065
model	regional	btree	hot_c	13.849 [0.140]	1.152 [0.257]	12.500 [0.300]	0.083	0.065
model	regional	isam	balanced	12.506 [0.341]	1.362 [0.118]	11.200 [0.250]	0.084	0.065
model	regional	isam	hot_c	13.852 [0.364]	1.280 [0.175]	12.500 [0.300]	0.083	0.065
model	regional	none	balanced	18.166 [0.295]	7.028 [0.228]	11.200 [0.250]	0.084	0.065
model	regional	none	hot_c	19.397 [0.176]	6.872 [0.302]	12.500 [0.300]	0.083	0.065

Tabla 17: Construcción del índice secundario (suma de tres nodos), espacio y encaminamiento.

Modo	Estrategia	Índice	Perfil	Build ms	Índice MiB	Ops remotas	Ops origen C
base	hash	isam	balanced	214.970 [4.607]	14.438	90 [2]	30 [7]
base	regional	isam	balanced	224.243 [17.382]	14.453	20 [0]	30 [7]
model	hash	btree	balanced	301.609 [23.951]	7.125	90 [2]	30 [7]
model	hash	btree	hot_c	294.892 [3.665]	7.125	91 [2]	80 [4]
model	hash	isam	balanced	227.960 [14.704]	14.438	90 [2]	30 [7]
model	hash	isam	hot_c	225.067 [6.964]	14.438	91 [2]	80 [4]
model	hash	none	balanced	0.000 [0.000]	0.000	90 [2]	30 [7]
model	hash	none	hot_c	0.000 [0.000]	0.000	91 [2]	80 [4]
model	regional	btree	balanced	288.454 [7.298]	7.125	20 [0]	30 [7]
model	regional	btree	hot_c	292.111 [12.291]	7.125	20 [0]	80 [4]
model	regional	isam	balanced	232.760 [17.164]	14.453	20 [0]	30 [7]
model	regional	isam	hot_c	220.602 [3.230]	14.453	20 [0]	80 [4]
model	regional	none	balanced	0.000 [0.000]	0.000	20 [0]	30 [7]
model	regional	none	hot_c	0.000 [0.000]	0.000	20 [0]	80 [4]

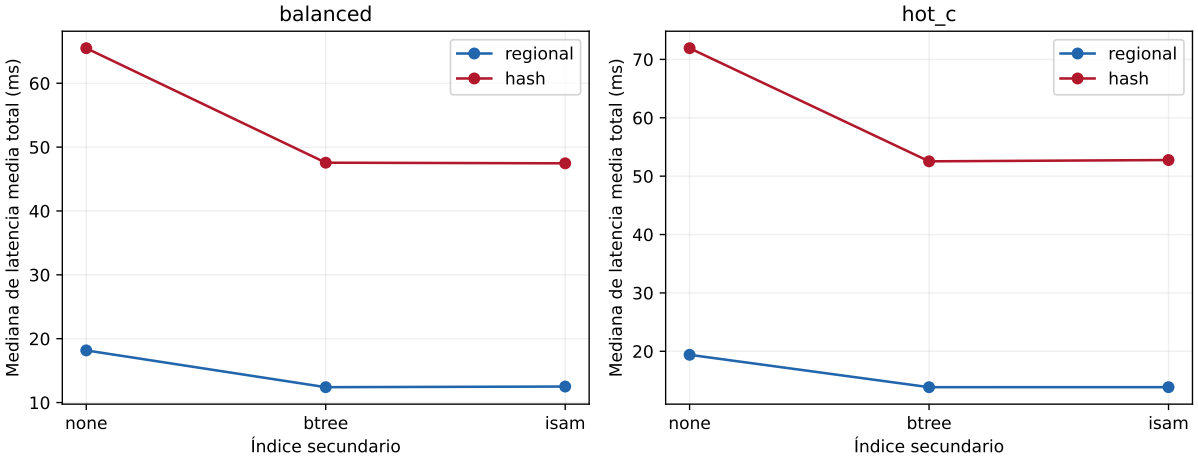


Figura 5: Tiempo total bajo el modelo aditivo de latencias. Incluye consultas reales y espera simulada.

Tabla 18: Desglose por operación: mediana [IQR] de la latencia media, en ms.

Modo	Estrategia	Índice	Perfil	Operación	SQL real	Total
base	hash	isam	balanced	customer	1.882 [0.039]	1.882 [0.039]
base	hash	isam	balanced	range	1.416 [0.097]	1.417 [0.097]
base	hash	isam	balanced	write	1.190 [0.167]	1.191 [0.167]
base	regional	isam	balanced	customer	0.836 [0.143]	0.838 [0.143]
base	regional	isam	balanced	range	1.595 [0.610]	1.597 [0.610]
base	regional	isam	balanced	write	0.925 [0.159]	0.925 [0.161]
model	hash	btree	balanced	customer	3.252 [0.241]	58.720 [0.951]
model	hash	btree	balanced	range	2.541 [0.233]	58.700 [1.792]
model	hash	btree	balanced	write	1.887 [0.119]	20.881 [3.644]
model	hash	btree	hot_c	customer	3.123 [0.088]	64.626 [2.126]
model	hash	btree	hot_c	range	2.324 [0.207]	64.936 [1.728]
model	hash	btree	hot_c	write	1.854 [0.137]	23.180 [4.149]
model	hash	isam	balanced	customer	3.062 [0.127]	58.459 [1.366]
model	hash	isam	balanced	range	2.369 [0.261]	58.534 [2.035]
model	hash	isam	balanced	write	1.914 [0.226]	20.551 [3.645]
model	hash	isam	hot_c	customer	3.214 [0.280]	64.643 [1.642]
model	hash	isam	hot_c	range	2.628 [0.204]	65.753 [1.527]
model	hash	isam	hot_c	write	1.952 [0.528]	23.840 [3.724]
model	hash	none	balanced	customer	38.937 [1.376]	95.390 [1.324]
model	hash	none	balanced	range	2.513 [0.190]	58.417 [1.299]
model	hash	none	balanced	write	1.831 [0.104]	20.866 [3.581]
model	hash	none	hot_c	customer	40.563 [0.711]	101.664 [1.360]
model	hash	none	hot_c	range	2.685 [0.294]	65.599 [0.795]
model	hash	none	hot_c	write	2.001 [0.080]	23.429 [3.929]
model	regional	btree	balanced	customer	0.906 [0.032]	0.907 [0.032]
model	regional	btree	balanced	range	2.620 [0.292]	58.316 [0.771]
model	regional	btree	balanced	write	0.971 [0.110]	0.971 [0.110]
model	regional	btree	hot_c	customer	0.822 [0.177]	0.822 [0.177]
model	regional	btree	hot_c	range	2.378 [0.501]	65.576 [1.463]
model	regional	btree	hot_c	write	0.914 [0.119]	0.914 [0.119]
model	regional	isam	balanced	customer	0.918 [0.082]	0.919 [0.083]
model	regional	isam	balanced	range	2.695 [0.131]	58.804 [1.088]
model	regional	isam	balanced	write	1.119 [0.190]	1.120 [0.190]
model	regional	isam	hot_c	customer	0.842 [0.207]	0.843 [0.207]
model	regional	isam	hot_c	range	2.591 [0.324]	65.248 [2.172]
model	regional	isam	hot_c	write	1.011 [0.150]	1.012 [0.150]
model	regional	none	balanced	customer	12.373 [0.179]	12.374 [0.179]
model	regional	none	balanced	range	2.484 [0.246]	58.467 [0.839]
model	regional	none	balanced	write	1.080 [0.066]	1.081 [0.066]
model	regional	none	hot_c	customer	12.020 [0.500]	12.022 [0.500]
model	regional	none	hot_c	range	2.582 [0.175]	65.460 [1.797]
model	regional	none	hot_c	write	1.021 [0.008]	1.021 [0.008]

Con B-tree, las medianas totales correspondientes fueron 12.406 y 47.559 ms. Estas diferencias pequeñas frente a ISAM no se presentan como significativas. Las lecturas por cliente son locales en la asignación regional y consultan tres fragmentos en hash; esa diferencia de encaminamiento forma parte de las estrategias comparadas. El perfil concentrado modifica el origen de las solicitudes y el costo de los enlaces utilizados, pero no demuestra saturación ni capacidad máxima bajo concurrencia.

Los índices secundarios B-tree sumaron una mediana de 7.125 MiB entre los tres nodos. ISAM sumó 14.453 MiB con asignación regional y 14.438 MiB con hash. Este escenario con claves de cliente repetidas tiene un comportamiento espacial distinto del experimento de claves únicas de la primera etapa. Se describe la observación sin atribuir toda la diferencia a un mecanismo interno que no se haya instrumentado de forma aislada.

Los planes normales registrados utilizaron el secundario en los 60 planes locales de la configuración ISAM y en los 60 de B-tree. El diagnóstico forzado se conserva aparte. Esto respalda el uso real del AM y permite distinguirlo del retraso simulado. Los catálogos pequeños conservan PK B-tree y las consultas por id utilizan esa capacidad en todas las alternativas.

11.2 Correctitud, réplica e indisponibilidad

La aceptación contrasta conteos, identificadores únicos, sumas y huellas con una referencia determinista. Comprueba localización, restricciones y una unión de cuatro tablas usando pedidos/detalles remotos y catálogos locales. Cada operación de carga verifica su resultado y el total de incrementos confirmados.

Se detiene realmente el contenedor C cuando no hay escrituras en vuelo. A/B mantienen sus lecturas locales y catálogos; una escritura local en A se confirma. El total global y una escritura hacia C deben fallar explícitamente, sin producir un total parcial como si estuviera completo. El contenedor se reinicia incluso si una aserción falla; la validación posterior comprueba la escritura confirmada y el contenido recuperado. Esta prueba no demuestra resolución de transacciones inciertas ni failover automático.

11.3 Discusión y límites del alcance

La conveniencia de una asignación depende de la consulta y del origen de sus solicitudes. La asignación regional puede reducir destinos remotos para una carga alineada con regiones; hash favorece reparto de filas, pero puede aumentar el número de fragmentos requeridos para una búsqueda por cliente. El cambio de concentración se interpreta dentro de este esquema sintético y cliente secuencial. No se extrapola a todo patrón de acceso ni se presenta un resultado de la Etapa I como medición distribuida.

Las réplicas pequeñas de catálogos mejoran disponibilidad de lecturas locales, con el compromiso de retraso hasta el refresco. El escenario no replica pedidos ni implementa consenso o transacciones globales. Los precios históricos de los detalles permanecen estables frente a actualizaciones de catálogo. Los datos y el SQL son reales; únicamente la matriz de latencias se modela.

12 Conclusiones de la entrega final

El proyecto integra un método de acceso ISAM en C con un escenario de tres servidores PostgreSQL y consultas mediante FDW. La comparación distribuida mantiene las mismas operaciones y permite relacionar fragmentación, ubicación, índices secundarios y destinos remotos. Las tablas y registros identifican las condiciones observadas; sus medidas centrales e IQR no implican superioridad universal de ISAM o de una asignación.

La validación hace visibles los límites de disponibilidad y consistencia: las consultas locales pueden continuar ante una interrupción ajena, mientras que los totales que requieren el fragmento detenido fallan. El refresco de snapshots permite observar tanto retraso como convergencia de catálogos. La defensa debe explicar estas decisiones y distinguir implementación real, simulación y limitaciones, además de poder ejecutar las pruebas reproducibles.

Contribución propia y recursos utilizados

Según la propuesta, el equipo desarrolla el layout ISAM, directorio, overflow, callbacks, generadores y harness. Se utilizan PostgreSQL y su interfaz oficial, GenericXLog, buffer manager, libc, GCC, make, Docker, Python, matplotlib y LaTeX. La propuesta identifica contrib/bloom y access/heap como referencias de convenciones, sin atribuirles el núcleo ISAM del proyecto.

La ampliación distribuida reutiliza PostgreSQL y postgres_fdw, Docker, psycopg, Python, matplotlib y LaTeX. Su diseño, scripts y redacción recibieron asistencia de OpenAI Codex. El Gantt es un antecedente de planificación; no acredita autoría individual de los cambios actuales.

La tabla 19 resume las responsabilidades del cronograma en las semanas 1–5.

Integrante	Tareas (numeración del Gantt)
Fabian	1.1 prototipo mínimo del AM en C; 2.5 `ambuild` inicial; 3.2 `aminsert`; 4.1 `ambeginscan`, `amrescan`, `amgettuple` y `amendscan`; 5.1 entorno Docker
Bryan	1.2 bloques y layout físico; 2.2 búsqueda; 3.1 inserción con overflow; 4.3 `amcostestimate` y callbacks obligatorios; 5.3 corrección de errores finales
Andy	1.3 índice de niveles; 2.1 construcción del índice; 3.5 pruebas de overflow con carga sesgada; 4.2 `CREATE ACCESS METHOD`; 5.3 corrección de errores finales
Jorge	1.4 referencias y propuesta inicial; 2.4 pruebas unitarias; 3.4 demo y documentación; 4.4 pruebas de integración; 5.4 preparación de defensa
Leonardo	definición de datasets D1/D2/D3; 2.3 generadores de datasets; 3.3 scripts de workload W1–W4; 4.5 benchmarking preliminar; 5.2 comparación preliminar con B-tree

Tabla 19: Responsabilidades por integrante según el cronograma, semanas 1–5.

A Correspondencia con la propuesta inicial, sección 5.5

Elemento requerido	Evidencia en esta entrega
Elección y motivación	Introducción; ISAM e igualdad/rango
Fuentes de calidad	Bibliografía académica y manual oficial citados en la propuesta
Representación y operaciones	Layout y construcción/búsqueda/inserción
Clave y consultas	int4, una columna, igualdad y rango
Integración PostgreSQL	IndexAmRoutine, opclass y callbacks
Componentes del equipo	Núcleo, adaptadores, generadores y harness identificados
Recursos externos	Declaración anterior y referencias de la propuesta

Riesgos y plan	Matriz de riesgos, semanas S1–S5 y bloques internos 0–6

La propuesta original de dos páginas se conserva en `docs/propuesta-inicial.pdf`. Este informe amplía la explicación técnica y los resultados; no sustituye ni pretende cumplir el límite de dos páginas del documento de Semana 1.

B Comandos reproducibles y defensa

```
make test
make integration
make crash
make crash-load
make bench       # 100000, 500000, 1000000; D1/D2/D3; 5 replicas
make overflow    # experimento adicional de concentracion
make demo
make report
make pdf
```

Las bases de benchmark y overflow son distintas de bd2. Los scripts no borran volúmenes al arrancar. Las ejecuciones oficiales y sus registros se conservan en `docs/results/`; la guía y preguntas de defensa están en `docs/defensa.md`. El código fuente y los scripts de prueba se encuentran en el repositorio del proyecto.

Repositorio: `https://github.com/lMafa1813l/BD2-Gestor-isam-postgres`.

Referencias

[1] R. Ramakrishnan y J. Gehrke. *Database Management Systems*, 3.ª edición. McGraw-Hill, 2003. Capítulos 8 y 10 citados en la propuesta inicial.

[2] A. Silberschatz, H. F. Korth y S. Sudarshan. *Database System Concepts*, 6.ª edición. McGraw-Hill, 2011.

[3] PostgreSQL Global Development Group. *PostgreSQL 18 Documentation: Index Access Method Interface*. `https://www.postgresql.org/docs/18/indexam.html`.

[4] Grupo 9, Bases de Datos II. *Propuesta Inicial – Proyecto BD II*. Documento entregado por el equipo, 2026. Copia en `docs/propuesta-inicial.pdf`.

[5] CS272, Bases de Datos II, 2026-II. *Proyecto BDII: Indexación y Distribución de Datos en PostgreSQL*. Pautas oficiales aportadas por el equipo, secciones 5–11.

[6] A. Silberschatz, H. F. Korth y S. Sudarshan. *Fundamentos de bases de datos*, 6.ª edición. McGraw-Hill Education, 2014.

[7] PostgreSQL Global Development Group. *PostgreSQL 18 Documentation: postgres_fdw*. `https://www.postgresql.org/docs/18/postgres-fdw.html`.